

SMC Fabric — Overview and Architecture

SMC Fabric — Overview and Architecture

1. Purpose
2. High-Level Block Diagram
3. Data-Path Flows
 - 3.1 Internal master → local peripheral
 - 3.2 System-AXI inbound (`sys_axi_in`)
 - 3.3 SEP AXI inbound (`sep_axi_in`)
 - 3.4 Internal master → system NoC (outbound)
4. Socket Routing Map
 - 4.1 Target sockets (inbound) → handler → path
 - 4.2 Local path → initiator socket (`route_local` / `local_decode`)
 - 4.3 Outbound path → `output_axi` (`route_outbound`)
 - 4.4 Internal CSR sockets serviced inside the fabric
5. Key Design Decisions
6. Temporal Decoupling Classification

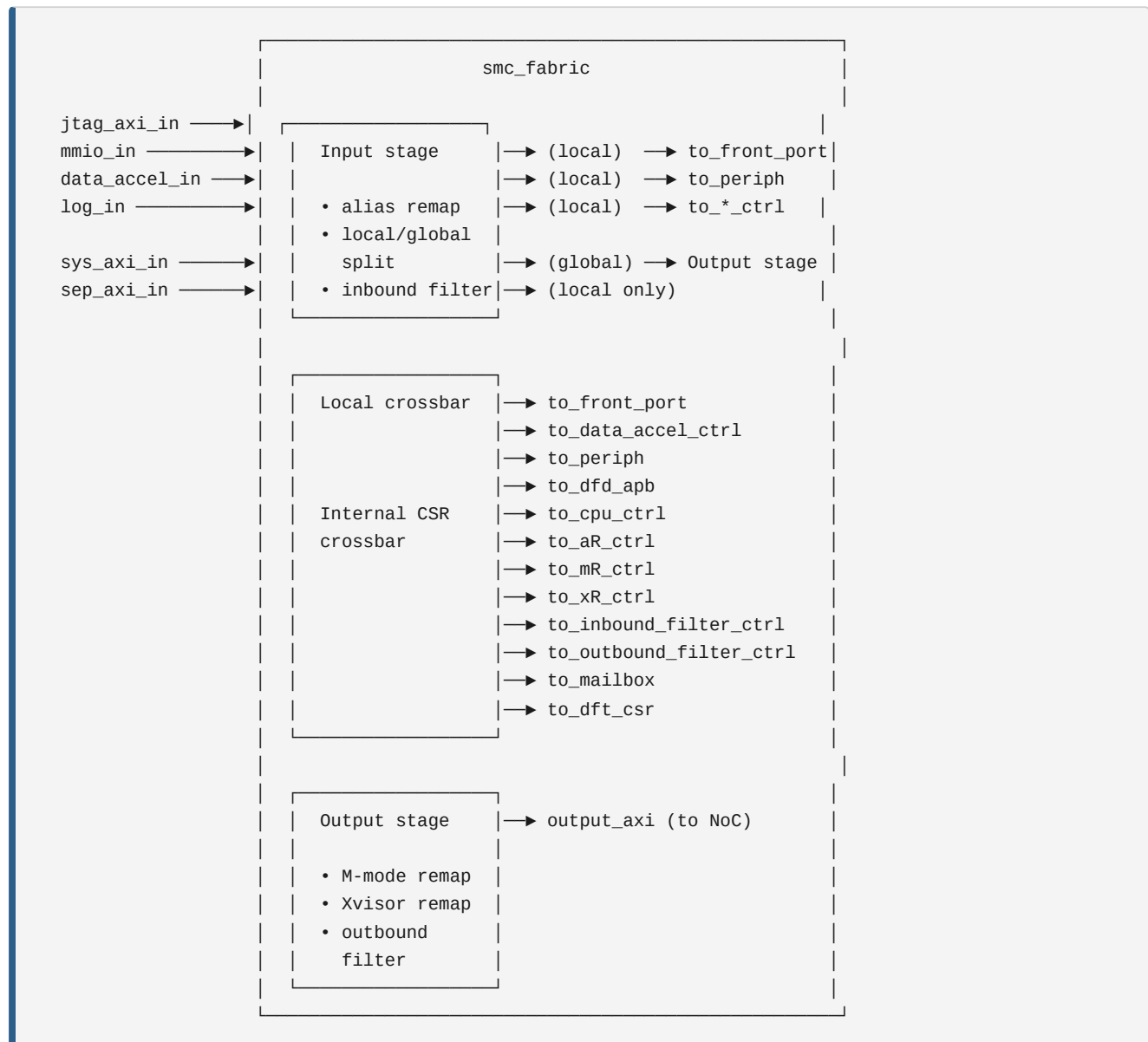
SMC Fabric — Overview and Architecture

1. Purpose

`smc_fabric` is the central interconnect of the System Management Controller (SMC). It implements:

- **Dual AXI network routing** — AXI4 64-bit high-performance (HP) path for memory/accelerator traffic and AXI4-Lite 32-bit low-performance (LP) path for peripheral register accesses.
 - **Alias remap** — 8 configurable regions that re-map internal master addresses before routing. Applied to MMIO, JTAG, Log, and DMA initiators.
 - **M-mode remap** — 8 configurable regions for M-mode privileged traffic destined to a separate NoC aperture.
 - **Xvisor remap** — 8 configurable regions for hypervisor (H-mode) traffic destined to yet another NoC aperture.
 - **Inbound filter** — 16-entry allow/deny filter on the system AXI input (`sys_axi_in`). Checks address range, 4-bit source ID, and NS bit. Default policy: deny all (allow-list).
 - **Outbound filter** — 16-entry allow/deny filter on the system AXI output (`output_axi`). Same policy engine; default policy: allow all (deny-list).
 - **Local/Global aperture model** — controlled by `LOCAL_BASE`, `GLOBAL_BASE`, and `REGION_SIZE`. Addresses inside the aperture are routed to the local crossbar; all others go to the outbound (NoC) path.
-

2. High-Level Block Diagram



3. Data-Path Flows

3.1 Internal master → local peripheral

```

CPU MMIO / JTAG / Log / DMA
  |
  ▼
[alias remap]   (8-entry table, shared across all internal initiators)
  |
  ▼
[local/global aperture check]
  | addr ∈ [LOCAL_BASE, +REGION_SIZE) | otherwise
  |   or [GLOBAL_BASE, +REGION_SIZE) |
  ▼                               ▼
[addr truncated to 32 bits]       Output stage
  |
  ▼
Local crossbar → address-range decode → downstream socket

```

3.2 System-AXI inbound (`sys_axi_in`)

```

sys_axi_in
  |
  ▼
[inbound filter]   16 entries, deny-by-default
  | pass           | deny
  ▼               ▼
[addr truncated 32 bits]   TLM_ADDRESS_ERROR_RESPONSE / 0xBADCAB1E
  |
  ▼
Local crossbar

```

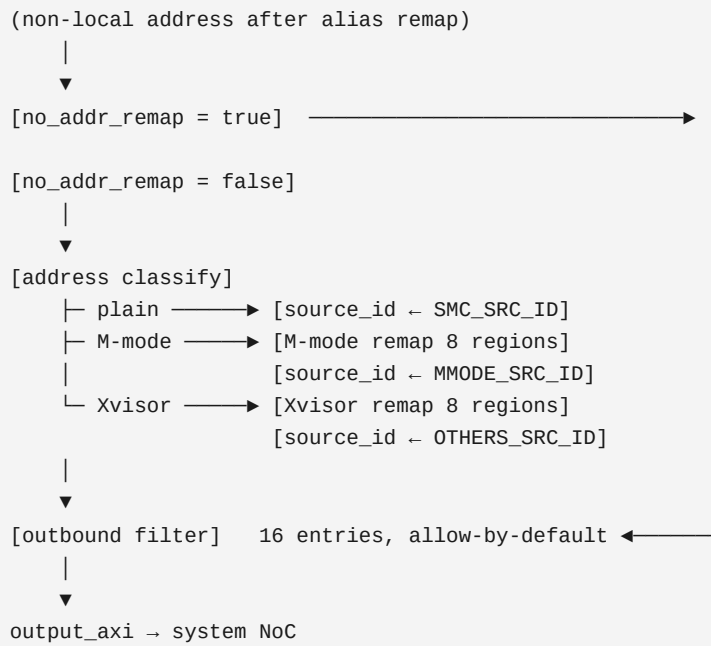
3.3 SEP AXI inbound (`sep_axi_in`)

```

sep_axi_in
  |
  ▼
[addr truncated 32 bits]
  |
  ▼
Local crossbar   (always local; never reaches outbound path)

```

3.4 Internal master → system NoC (outbound)



4. Socket Routing Map

This section maps every TLM socket to the route it carries. Sockets are declared in `include/smc_fabric.h`; the routing logic lives in `src/smc_fabric.cpp` (`bt_*` handlers, `route_local`, `local_decode`, `route_outbound`).

4.1 Target sockets (inbound) → handler → path

Target socket	b_transport handler	Route
jtag_axi_in	bt_jtag → bt_internal	alias remap → is_local? → local or outbound
mmio_in	bt_mmio → bt_internal	alias remap → is_local? → local or outbound
data_accel_in	bt_data_accel → bt_internal	alias remap → is_local? → local or outbound
log_in	bt_log → bt_internal	alias remap → is_local? → local or outbound
sys_axi_in	bt_sys_axi	inbound filter → pass: truncate → local only / deny: error
sep_axi_in	bt_sep_axi	truncate → local only (no filter, no remap)

sys_axi_in and sep_axi_in are local-only by construction; they never reach the outbound path.

4.2 Local path → initiator socket (route_local / local_decode)

Address offsets are relative to LOCAL_BASE (lower 25 bits of the masked 32-bit address).

Absolute address ranges (verbatim from smc_local_xbar_pkg.sv / smc_internal_axi_lite_xbar_pkg.sv;
LOCAL_BASE = 0xC000_0000).

Absolute range	Destination	Contents
0xC000_F800 – 0xC000_FFFF	to_dft_csr	DFT CSRs
0xC001_0000 – 0xC001_1FFF	internal handle_global_csr , else to_cpu_ctrl	GLOBAL/LOCAL_BASE, REGION_SIZE, else CPU ctrl
0xC001_2000 – 0xC001_2FFF	internal (alias table)	ALIAS_REMAP CSRs
0xC001_3000 – 0xC001_3FFF	internal (mmode table)	MMODE_REMAP CSRs
0xC001_4000 – 0xC001_4FFF	internal (xvisor table)	XVISOR_REMAP CSRs
0xC001_5000 – 0xC001_5FFF	internal (inbound filter)	inbound filter entries
0xC001_6000 – 0xC001_6FFF	internal (outbound filter)	outbound filter entries
0xC001_8000 – 0xC003_7FFF	to_mailbox	Mailbox
0xC000_0000 – 0xC000_0FFF	to_front_port	WDT / debug
0xC004_0000 – 0xC015_FFFF	to_front_port	Scratchpad memory (SPM)
0xC400_0000 – 0xC7FF_FFFF	to_front_port	PLIC (outside alias aperture)
0xC800_0000 – 0xC801_FFFF	to_front_port	CLINT / BEU (outside alias aperture)
0xC003_8000 – 0xC003_FFFF	to_data_accel_ctrl	DMA + zeroer ctrl
0xC000_2000 – 0xC000_E7FF	to_periph	peripheral main (UART / I2C / GPIO / ...)
0xC040_0000 – 0xC07F_FFFF	to_periph	peripheral extended
0xC016_0000 – 0xC025_FFFF	to_dfd_apb	DFD registers (APB)
any other local address	—	TLM_ADDRESS_ERROR_RESPONSE (deny)

CSR rows marked *internal* are serviced inside the fabric (tables updated directly); the matching `to_*_ctrl` sockets exist only so a testbench may bind observation stubs.

4.3 Outbound path → `output_axi` (`route_outbound`)

All non-local traffic from the four internal masters converges on the single `output_axi` socket.

Condition	source_id tag	Destination
no_addr_remap = true (default)	unchanged	output_axi
plain (non-M-mode, non-Xvisor)	SMC_SRC_ID	output_axi
M-mode window	MMODE_SRC_ID	output_axi
Xvisor window	OTHERS_SRC_ID	output_axi
outbound filter deny	—	error (no forward)

4.4 Internal CSR sockets serviced inside the fabric

`to_a_ctrl`, `to_mR_ctrl`, `to_xR_ctrl`, `to_inbound_filter_ctrl`, and `to_outbound_filter_ctrl` are handled inside the fabric (their tables are updated directly); the sockets exist for optional testbench observation. `to_dft_csr`, `to_mailbox`, and `to_dfd_apb` are reachable through the decode above and forward

downstream.

5. Key Design Decisions

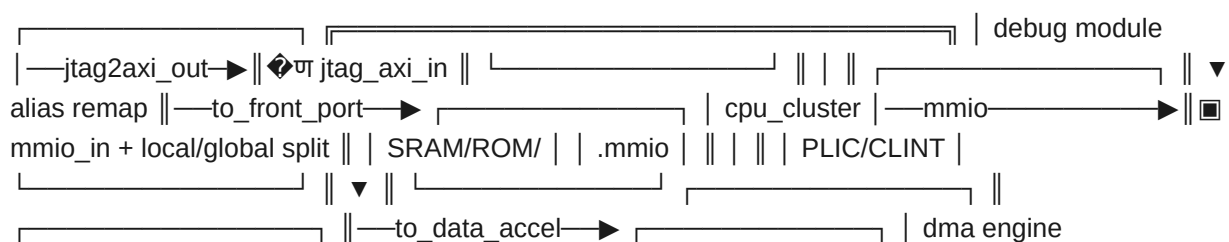
Feature	LT modelling strategy
Alias remap	<code>std::array<alias_region, 8></code> walked on every transaction; table rebuilt on CSR write
Inbound / outbound filter	Bit-exact <code>filter_ctrl.rdl</code> image (<code>FILTER_CONFIG / START_ADDR / END_ADDR</code>). Lowest-index <i>hit</i> wins: <code>addr_mode</code> (enable) + inclusive <code>[start,end]</code> range + exact <code>src_id</code> (<code>0</code> $\Rightarrow$ ignored) + exact NS match (<code>tx_ns == allow_ns</code>), then <code>read_en/write_en</code> decides. No hit $\rightarrow$ <code>!BlockByDefault</code> (inbound deny, outbound allow). Deny $\rightarrow$ <code>TLM_ADDRESS_ERROR_RESPONSE + 0xBADCAB1E</code>
M-mode / Xvisor output remap	Bit-exact <code>output_remap</code> image: 8 fixed 1 MB regions, <code>index = (addr-base)[22:20]</code> , <code>out = {REGION_ATTRS.offset[55:20], (addr-base)[19:0]}</code> (no valid bit); <code>smc_axi_extension.source_id</code> overridden per path
Arbitration (mux / demux)	Not modelled — <code>b_transport</code> is cooperatively serialised by the SystemC scheduler
ID width conversions	Transparent in LT — <code>smc_axi_extension.axi_id</code> passes through unchanged
Clock gating	Not modelled — always-on in LT
Output remap bypass	Constructor parameter <code>no_addr_remap = true</code> ; skips M-mode and Xvisor tables entirely

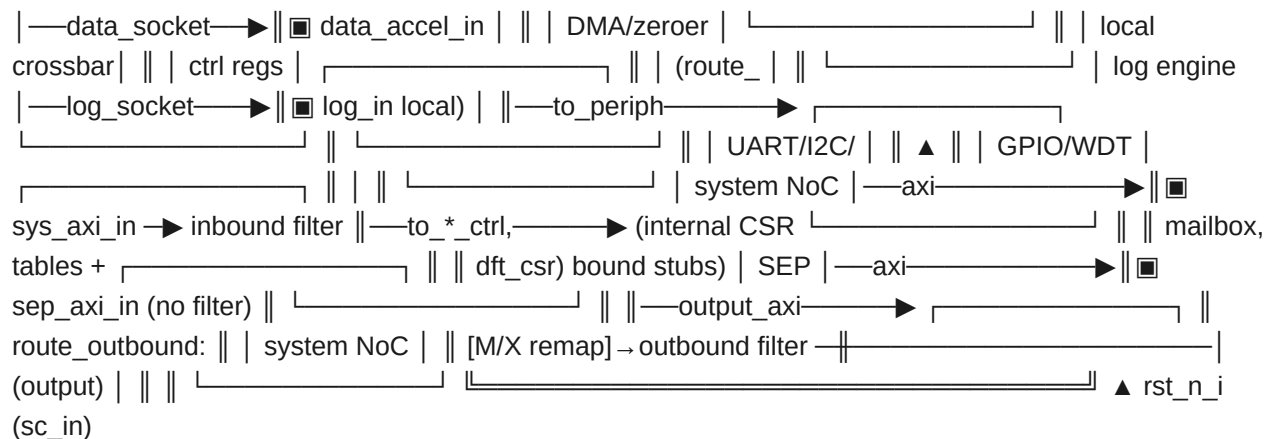
6. Temporal Decoupling Classification

`smc_fabric` is **Bucket D — pure target**.

- No `t1m_quantumkeeper` owned.
- Every `b_transport` handler adds `reg_access_ns` to the incoming `delay` and returns; routing itself is zero-latency.
- Initiators polling fabric configuration registers must apply an **S4 sync** before the poll for coherence with concurrent writers.
- DMI is **never** advertised by the fabric. After any write to `GLOBAL_BASE`, `REGION_SIZE`, or any remap/filter register, the fabric calls `invalidate_direct_mem_ptr` on all its initiator sockets so downstream DMI consumers (e.g. scratchpad memory) re-fetch their pointers.

INITIATORS `smc_fabric` (SC_MODULE) TARGETS (initiator socket) (target socket) (initiator socket) (target socket)





Legend: = `simple_target_socket<...,64>` —name—> = `b_transport()` call direction all sockets

BUSWIDTH = 64 (response returns on the same call)